

```
from google.colab import drive  
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call dr

```
!ls "/content/drive/MyDrive"
```

```
ViolenciadeGenero.gdoc
"we didn't start the fire.gdoc"
'WhatsApp Image 2023-02-09 at 22.00.11.jpeg'
'WhatsApp Image 2023-06-21 at 14.26.12.jpeg'
'WhatsApp Image 2023-07-27 at 10.00.27.jpeg'
'WhatsApp Video 2025-05-28 at 21.50.56 (1).mp4'
'WhatsApp Video 2025-05-28 at 21.50.56.mp4'
'Workshop - Kinematic variables & articular movement [Editable].docx'
'Writing Assignment - Interview (Stan Lee).gdoc'
zacatlan.gdoc
zimmer.gdoc
```

```
!find "/content/drive/MyDrive" -type d -name "BUS-UCLM Breast ultrasound lesior
```

```
/content/drive/MyDrive/BUS-UCLM Breast ultrasound lesion segmentation dataset
/content/drive/MyDrive/BUS-UCLM Breast ultrasound lesion segmentation dataset/BU
```

```
%cd /content
!git clone https://github.com/noeliavallez/BUS-UCLM-Dataset.git
```

```
/content
fatal: destination path 'BUS-UCLM-Dataset' already exists and is not an empty di
```

```
%cd /content/BUS-UCLM-Dataset
```

```
/content/BUS-UCLM-Dataset
```

```
!pip install -q opencv-python scikit-image pandas matplotlib tqdm scipy
```

```
from pathlib import Path
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from tqdm import tqdm
from skimage.filters import threshold_otsu
from skimage.morphology import (
    disk,
    opening,
    closing,
    remove_small_objects,
    remove_small_holes
)
from skimage.measure import label, regionprops
from scipy import ndimage as ndi
```

```
DATA_ROOT = Path("/content/BUS-UCLM-Dataset/data/BUS-UCLM")
IMG_DIR = DATA_ROOT / "images"
MASK_DIR = DATA_ROOT / "masks"

SIZE = (512, 512) # ancho, alto para OpenCV

image_paths = sorted(list(IMG_DIR.glob("*.png")))

print("IMG_DIR existe:", IMG_DIR.exists())
print("MASK_DIR existe:", MASK_DIR.exists())
print("Número de imágenes:", len(image_paths))
print("Número de máscaras:", len(list(MASK_DIR.glob('*.png'))))

assert IMG_DIR.exists(), "No existe la carpeta de imágenes."
assert MASK_DIR.exists(), "No existe la carpeta de máscaras."
assert len(image_paths) > 0, "No se encontraron imágenes."
```

```
IMG_DIR existe: True
MASK_DIR existe: True
Número de imágenes: 683
Número de máscaras: 683
```

```
def read_image(image_path):
    img_bgr = cv2.imread(str(image_path), cv2.IMREAD_COLOR)

    if img_bgr is None:
        raise ValueError(f"No se pudo leer la imagen: {image_path}")

    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
    return gray

def read_gt_mask(mask_path, target_size=SIZE):
    """
    Lee la máscara de BUS-UCLM.

    En BUS-UCLM normalmente:
    - Verde = lesión benigna
    - Rojo = lesión maligna
    - Negro = fondo

    Devuelve:
    - binary_mask: 1 donde hay lesión, 0 fondo
    - class_mask: 1 benigna, 2 maligna, 0 fondo
    """

    mask = cv2.imread(str(mask_path), cv2.IMREAD_UNCHANGED)

    if mask is None:
        raise ValueError(f"No se pudo leer la máscara: {mask_path}")
```

```

if mask.ndim == 2:
    binary_mask = mask > 0
    class_mask = np.zeros(mask.shape, dtype=np.uint8)
    class_mask[binary_mask] = 1

else:
    if mask.shape[2] == 4:
        mask = mask[:, :, :3]

    b, g, r = cv2.split(mask)

    class_mask = np.zeros(mask.shape[:2], dtype=np.uint8)

    benign = (g > 80) & (g > r) & (g > b)
    malignant = (r > 80) & (r > g) & (r > b)

    class_mask[benign] = 1
    class_mask[malignant] = 2

    # Fallback por si la máscara tiene antialiasing o colores no puros
    any_color = (r > 30) | (g > 30) | (b > 30)
    class_mask[(class_mask == 0) & any_color] = 1

    binary_mask = class_mask > 0

binary_mask = binary_mask.astype(np.uint8)
class_mask = class_mask.astype(np.uint8)

if target_size is not None:
    binary_mask = cv2.resize(
        binary_mask,
        target_size,
        interpolation=cv2.INTER_NEAREST
    )

    class_mask = cv2.resize(
        class_mask,
        target_size,
        interpolation=cv2.INTER_NEAREST
    )

return binary_mask.astype(np.uint8), class_mask.astype(np.uint8)

```

```

def preprocess_image(gray, target_size=SIZE):
    """
    Preprocesamiento para ultrasonido mamario:
    1. Redimensiona a 512x512.
    2. Reduce ruido speckle con Non-Local Means.
    3. Mejora contraste local con CLAHE.
    4. Aplica corrección gamma ligera.

```

```

"""

resized = cv2.resize(gray, target_size, interpolation=cv2.INTER_LINEAR)

if resized.dtype != np.uint8:
    resized = cv2.normalize(
        resized,
        None,
        0,
        255,
        cv2.NORM_MINMAX
    ).astype(np.uint8)

denoised = cv2.fastNlMeansDenoising(
    resized,
    None,
    h=8,
    templateWindowSize=7,
    searchWindowSize=21
)

clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)

enhanced = clahe.apply(denoised)

gamma = 0.9
enhanced_float = enhanced.astype(np.float32) / 255.0
gamma_corrected = np.power(enhanced_float, gamma)

result = (gamma_corrected * 255).astype(np.uint8)

return result

```

```

def make_ultrasound_roi(gray_resized):
    """
    Crea una región de interés para evitar:
    - Bordes negros.
    - Letras o marcas de la imagen.
    - Zona inferior donde suelen aparecer sombras acústicas.
    """

    h, w = gray_resized.shape

    roi = gray_resized > 8

    roi = closing(roi.astype(bool), disk(8))
    roi = remove_small_objects(roi, min_size=2000)

```

```

roi = ndi.binary_fill_holes(roi)

labeled = label(roi)
regions = regionprops(labeled)

if len(regions) > 0:
    largest = max(regions, key=lambda r: r.area)
    roi = labeled == largest.label
else:
    roi = np.ones_like(gray_resized, dtype=bool)

# Recortes para evitar zonas problemáticas
roi[:int(0.05 * h), :] = False
roi[int(0.88 * h):, :] = False
roi[:, :int(0.03 * w)] = False
roi[:, int(0.97 * w):] = False

return roi.astype(bool)

```

```

def select_best_component(candidate, gray_resized, enhanced, local_dark, roi, ε):
    """
    Selecciona la región más parecida a una lesión.

    Criterios:
    - Área suficiente.
    - Región oscura.
    - Contraste local oscuro alto.
    - Forma compacta.
    - No demasiado alargada.
    - No estar demasiado abajo.
    """

    labeled = label(candidate)
    regions = regionprops(labeled)

    if len(regions) == 0:
        return np.zeros_like(candidate, dtype=np.uint8), []

    h, w = candidate.shape
    image_area = h * w

    if expected_area is None:
        expected_area = 9000 # aproximado útil para 512x512

    center_y, center_x = h / 2, w / 2
    max_dist = np.sqrt(center_y**2 + center_x**2)

    scored_regions = []

    for region in regions:

```

```
area = region.area

# Aquí está la parte importante:
# ignoramos migajas de ruido y regiones absurdamente grandes.
if area < 1500:
    continue

if area > 45000:
    continue

minr, minc, maxr, maxc = region.bbox
height = maxr - minr
width = maxc - minc

if height == 0 or width == 0:
    continue

aspect_ratio = width / height

# Lesión puede ser ovalada, pero no debería ser una línea.
if aspect_ratio < 0.30 or aspect_ratio > 3.50:
    continue

cy, cx = region.centroid

# Evita escoger sombras muy inferiores
if cy > 0.80 * h:
    continue

coords = region.coords
rr = coords[:, 0]
cc = coords[:, 1]

mean_raw = np.mean(gray_resized[rr, cc]) / 255.0
mean_enhanced = np.mean(enhanced[rr, cc]) / 255.0
mean_local_dark = np.mean(local_dark[rr, cc]) / 255.0

darkness = 1.0 - mean_raw
enhanced_darkness = 1.0 - mean_enhanced

solidity = region.solidity
eccentricity = region.eccentricity

perimeter = region.perimeter if region.perimeter > 0 else 1
circularity = 4 * np.pi * area / (perimeter ** 2)

dist_center = np.sqrt((cy - center_y)**2 + (cx - center_x)**2)
dist_center_norm = dist_center / max_dist

area_score = np.exp(-abs(area - expected_area) / expected_area)
```

```

        score = (
            2.5 * darkness +
            2.0 * enhanced_darkness +
            2.0 * mean_local_dark +
            2.2 * area_score +
            1.2 * solidity +
            1.0 * circularity -
            0.8 * eccentricity -
            0.4 * dist_center_norm
        )

        scored_regions.append({
            "label": region.label,
            "score": score,
            "area": area,
            "centroid_y": cy,
            "centroid_x": cx,
            "aspect_ratio": aspect_ratio,
            "solidity": solidity,
            "eccentricity": eccentricity,
            "circularity": circularity,
            "darkness": darkness,
            "local_dark": mean_local_dark
        })

    if len(scored_regions) == 0:
        return np.zeros_like(candidate, dtype=np.uint8), []

    best = max(scored_regions, key=lambda x: x["score"])
    best_label = best["label"]

    final_mask = labeled == best_label

    final_mask = closing(final_mask.astype(bool), disk(8))
    final_mask = remove_small_holes(final_mask, area_threshold=1800)
    final_mask = ndi.binary_fill_holes(final_mask)

    return final_mask.astype(np.uint8), scored_regions

```

```

def segment_lesion(gray, expected_area=90000):
    """
    Segmentación automática de lesión en imágenes de ultrasonido.

    La función hace varios intentos con diferentes umbrales de
    Esto ayuda porque no todas las imágenes tienen la misma
    """

    gray_resized = cv2.resize(gray, SIZE)
    enhanced = preprocess_image(gray)

    roi = make_ultrasound_roi(gray_resized)

```

```

# Fondo local: imagen suavizada.
background = cv2.GaussianBlur(
    enhanced,
    ksize=(0, 0),
    sigmaX=25,
    sigmaY=25
)

# Contraste local oscuro:
# si background - enhanced es alto,
local_dark = background.astype(np.int32) - enhanced
local_dark[local_dark < 0] = 0

local_dark = cv2.normalize(
    local_dark,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

roi_gray = enhanced[roi]
roi_dark = local_dark[roi]

if roi_gray.size == 0:
    roi_gray = enhanced.flatten()
    roi_dark = local_dark.flatten()

all_candidates = []

# Probamos varios umbrales porque el
dark_percentiles = [30, 35, 40, 45, 50, 55]
local_dark_percentiles = [45, 50, 55, 60, 65, 70]

best_mask = np.zeros_like(gray_resized)
best_score = -np.inf
best_candidate = np.zeros_like(gray_resized)
best_regions = []

for dp in dark_percentiles:
    for lp in local_dark_percentiles:

        dark_thr = np.percentile(roi_gray, dp)
        local_thr = np.percentile(roi_dark, lp)

        candidate = (
            roi &
            (enhanced < dark_thr) &
            (local_dark > local_thr)
        )

```

```

        candidate = closing(candidate)
        candidate = opening(candidate)
        candidate = remove_small_objects(candidate)
        candidate = remove_small_holes(candidate)

    mask, regions = select_best_regions(
        candidate=candidate,
        gray_resized=gray_resized,
        enhanced=enhanced,
        local_dark=local_dark,
        roi=roi,
        expected_area=expected_area
    )

    if len(regions) == 0:
        continue

    current_best_region = max(regions, key=lambda r: r["score"])

    if current_best_region["score"] > best_score:
        best_score = current_best_region["score"]
        best_mask = mask.copy()
        best_candidate = candidate
        best_regions = regions.copy()

    all_candidates.extend(regions)

# Fallback: si no encontró nada, usar el ROI
if best_mask.sum() == 0:
    dark_thr = np.percentile(roi_gray, 95)

    candidate = roi & (enhanced < dark_thr)

    candidate = closing(candidate.astype(np.uint8))
    candidate = opening(candidate.astype(np.uint8))
    candidate = remove_small_objects(candidate)
    candidate = remove_small_holes(candidate)

    best_mask, best_regions = select_best_regions(
        candidate=candidate,
        gray_resized=gray_resized,
        enhanced=enhanced,
        local_dark=local_dark,
        roi=roi,
        expected_area=expected_area
    )

    best_candidate = candidate.astype(np.uint8)

debug = {
    "gray_resized": gray_resized,

```

```

        "preprocessed": enhanced,
        "roi": roi.astype(np.uint8),
        "local_dark": local_dark,
        "candidate": best_candidate.astype(np.uint8),
        "regions": best_regions,
        "all_candidates": all_candidates
    }

```

```

    return best_mask.astype(np.uint8), c

```

```

def compute_metrics(pred_mask, gt_mask):
    """
    Calcula métricas asegurando que predicción y GT tengan el mismo tamaño.
    """

    pred = pred_mask.astype(np.uint8)
    gt = gt_mask.astype(np.uint8)

    if pred.shape != gt.shape:
        gt = cv2.resize(
            gt,
            (pred.shape[1], pred.shape[0]),
            interpolation=cv2.INTER_NEAREST
        )

    pred = pred.astype(bool)
    gt = gt.astype(bool)

    TP = np.logical_and(pred, gt).sum()
    FP = np.logical_and(pred, np.logical_not(gt)).sum()
    FN = np.logical_and(np.logical_not(pred), gt).sum()
    TN = np.logical_and(np.logical_not(pred), np.logical_not(gt)).sum()

    iou = TP / (TP + FP + FN + 1e-8)
    dice = (2 * TP) / (2 * TP + FP + FN + 1e-8)
    sensitivity = TP / (TP + FN + 1e-8)
    accuracy = (TP + TN) / (TP + TN + FP + FN + 1e-8)
    precision = TP / (TP + FP + 1e-8)

    return {
        "IoU": iou,
        "Dice": dice,
        "Sensitivity": sensitivity,
        "Accuracy": accuracy,
        "Precision": precision,
        "TP": int(TP),
        "FP": int(FP),
        "FN": int(FN),
        "TN": int(TN)
    }

```

```
def find_sample_with_lesion(image_paths, min_gt_area=300):
    """
    Busca automáticamente una imagen que sí tenga lesión.
    Así evitamos probar con una imagen normal por accidente.
    """

    for img_path in image_paths:
        mask_path = MASK_DIR / img_path.name

        if not mask_path.exists():
            continue

        gt_binary, gt_class = read_gt_mask(mask_path)

        if gt_binary.sum() >= min_gt_area:
            return img_path

    raise ValueError("No se encontró una imagen con lesión suficiente.")
```

```
sample_img_path = find_sample_with_lesion(image_paths)
sample_mask_path = MASK_DIR / sample_img_path.name

print("Imagen de prueba:", sample_img_path.name)
```

Imagen de prueba: ALWI_000.png

```
def plot_segmentation_result(gray, gt_binary, pred_mask, debug, metrics):
    """
    Visualiza:
    1. Imagen original.
    2. Imagen preprocesada.
    3. Contraste local oscuro.
    4. Candidatos.
    5. Máscara predicha.
    6. Overlay GT vs Pred.
    """

    gray_resized = cv2.resize(gray, SIZE, interpolation=cv2.INTER_LINEAR)

    if gt_binary.shape != pred_mask.shape:
        gt_binary = cv2.resize(
            gt_binary,
            (pred_mask.shape[1], pred_mask.shape[0]),
            interpolation=cv2.INTER_NEAREST
        )

    overlay = cv2.cvtColor(gray_resized, cv2.COLOR_GRAY2RGB)

    intersection = np.logical_and(pred_mask == 1, gt_binary == 1)
    only_gt = np.logical_and(pred_mask == 0, gt_binary == 1)
```

```

only_pred = np.logical_and(pred_mask == 1, gt_binary == 0)

overlay[intersection] = [255, 255, 0] # amarillo = coincidencia
overlay[only_gt] = [0, 255, 0] # verde = GT no detectado
overlay[only_pred] = [255, 0, 0] # rojo = falso positivo

plt.figure(figsize=(22, 4))

plt.subplot(1, 6, 1)
plt.imshow(gray_resized, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1, 6, 2)
plt.imshow(debug["preprocessed"], cmap="gray")
plt.title("Preprocesada")
plt.axis("off")

plt.subplot(1, 6, 3)
plt.imshow(debug["local_dark"], cmap="gray")
plt.title("Contraste local oscuro")
plt.axis("off")

plt.subplot(1, 6, 4)
plt.imshow(debug["candidate"], cmap="gray")
plt.title("Candidatos")
plt.axis("off")

plt.subplot(1, 6, 5)
plt.imshow(pred_mask, cmap="gray")
plt.title("Máscara predicha")
plt.axis("off")

plt.subplot(1, 6, 6)
plt.imshow(overlay)
plt.title(f"GT vs Pred\nDice={metrics['Dice']:.3f}, IoU={metrics['IoU']:.3f}")
plt.axis("off")

plt.tight_layout()
plt.show()

```

```

gray = read_image(sample_img_path)
gt_binary, gt_class = read_gt_mask(sample_img_path)

# Puedes ajustar expected_area.
# Para lesiones más grandes prueba 12000
pred_mask, debug = segment_lesion(
    gray,
    expected_area=12000
)

```

```

metrics = compute_metrics(pred_mask, gt_binary)

print("Archivo:", sample_img_path.name)
print("Área GT:", int(gt_binary.sum()))
print("Área predicha:", int(pred_mask.sum()))
print("Ratio lesión/imagen:", gt_binary.sum() / pred_mask.sum())

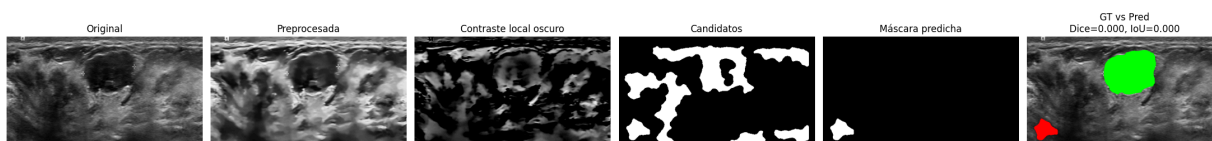
print("\nMétricas:")
for k, v in metrics.items():
    if k not in ["TP", "FP", "FN", "TN"]:
        print(f"{k}: {v:.4f}")

plot_segmentation_result(
    gray=gray,
    gt_binary=gt_binary,
    pred_mask=pred_mask,
    debug=debug,
    metrics=metrics
)

```

Archivo: ALWI_000.png
 Área GT: 12784
 Área predicha: 2008
 Ratio lesión/imagen: 0.04876708984375

Métricas:
 IoU: 0.0000
 Dice: 0.0000
 Sensitivity: 0.0000
 Accuracy: 0.9436
 Precision: 0.0000



```

results = []

for img_path in tqdm(image_paths):
    mask_path = MASK_DIR / img_path.name

    if not mask_path.exists():
        continue

    gray = read_image(img_path)
    gt_binary, gt_class = read_gt_mask(n

    pred_mask, debug = segment_lesion(gray)

    metrics = compute_metrics(pred_mask,

    gt_area = int(gt_binary.sum())
    pred_area = int(pred_mask.sum())

```

```

if gt_area == 0:
    lesion_type = "normal"

    # Para imágenes normales, Dice/IoU
    metrics["IoU"] = np.nan
    metrics["Dice"] = np.nan
    metrics["Sensitivity"] = np.nan

else:
    if np.any(gt_class == 2):
        lesion_type = "malignant"
    else:
        lesion_type = "benign"

row = {
    "filename": img_path.name,
    "lesion_type": lesion_type,
    "gt_area": gt_area,
    "pred_area": pred_area
}

row.update(metrics)
results.append(row)

df_results = pd.DataFrame(results)

print("Imágenes evaluadas:", len(df_results))
display(df_results.head())

```

100%|██████████| 683/683 [11:04<00:00, 1.03it/s]Imágenes evaluadas: 683

	filename	lesion_type	gt_area	pred_area	IoU	Dice	Sensitivity	Accuracy
0	ALWI_000.png	benign	12784	1153	0.0	0.0	0.0	0.9468
1	ALWI_001.png	normal	0	948	NaN	NaN	NaN	0.9963
2	ALWI_002.png	benign	3546	1676	0.0	0.0	0.0	0.9800
3	ALWI_003.png	benign	11423	855	0.0	0.0	0.0	0.9531
4	ALWI_004.png	benign	11450	855	0.0	0.0	0.0	0.9530

```

lesion_df = df_results[df_results["gt_area"] > 0].copy()
normal_df = df_results[df_results["gt_area"] == 0].copy()

print("Total imágenes:", len(df_results))
print("Imágenes con lesión:", len(lesion_df))
print("Imágenes normales:", len(normal_df))

```

```

print("\nMétricas promedio en imágenes con lesión:")
display(
    lesion_df[["IoU", "Dice", "Sensitivity", "Accuracy", "Precision"]]
    .mean()
    .to_frame("mean")
)

print("\nMétricas por tipo de lesión:")
display(
    lesion_df
    .groupby("lesion_type")[["IoU", "Dice", "Sensitivity", "Accuracy", "Precision"]]
    .mean()
)

print("\nÁrea predicha promedio en imágenes normales:")
print(normal_df["pred_area"].mean())

```

Total imágenes: 683
 Imágenes con lesión: 264
 Imágenes normales: 419

Métricas promedio en imágenes con lesión:

	mean 
IoU	0.047626
Dice	0.064867
Sensitivity	0.051804
Accuracy	0.919982
Precision	0.171698

Métricas por tipo de lesión:

	IoU	Dice	Sensitivity	Accuracy	Precision
lesion_type					
benign	0.036972	0.048610	0.042767	0.926956	0.112561
malignant	0.068222	0.096298	0.069275	0.906499	0.286029

Área predicha promedio en imágenes normales:
 2310.0167064439142

```

output_csv = "/content/BUS_UCLM_resultados_segimentacion.csv"
df_results.to_csv(output_csv, index=False)

print("Resultados guardados en:", output_csv)

```

Resultados guardados en: /content/BUS_UCLM_resultados_segmentacion.csv